

Talking to an LLM from Your Application

Coder Army

1. ChatGPT Is More Than an LLM

ChatGPT and an LLM are not the same thing.

When we open ChatGPT and write:

```
Explain Docker to me like I am a beginner.
```

we are not directly interacting with a transformer model.

There is an entire software application sitting between the user and the underlying model.

Examples of such applications include:

- ChatGPT
- Claude
- Gemini
- DeepSeek

The important distinction is:

An LLM is a capability. A GenAI application is software built around that capability.

A Simple Analogy

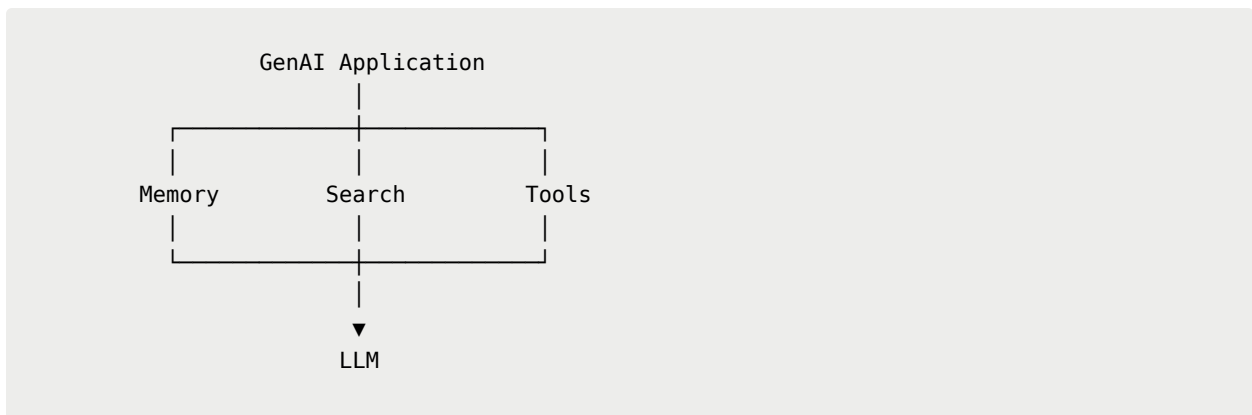
Think of an **LLM as an engine**.

An engine generates power, but an engine alone is not a car.

A complete car also needs:

- Steering
- Brakes
- Transmission
- Dashboard
- Sensors
- Navigation
- Safety systems

Similarly, an LLM can be the intelligence engine inside a much larger GenAI application.



The **LLM provides the language intelligence**, while the surrounding application determines what information reaches the model and what happens with its response.

This distinction is especially important for a Forward Deployed Engineer because much of GenAI engineering is about controlling this flow of information.

2. ChatGPT vs a Raw LLM

Consider the question:

What is the weather in Delhi right now?

A raw LLM and an AI application with external tools can handle this very differently.

Raw LLM

```
graph TD
    A[User Prompt] --> B[LLM]
    B --> C[Generated Answer]
```

A raw model can only generate an answer based on the information available to it in the request and what it learned during training.

It does not automatically have access to live weather information.

GenAI Application with Tools

A GenAI application can introduce another step:

```
graph TD
    A[User] --> B[Application]
    B --> C[Detects that live information is required]
    C --> D[Weather API / Search / Tool]
    D --> E[Current Weather Data]
    E --> F[LLM]
    F --> G[Natural-Language Answer]
    G --> H[User]
```

The **ability to generate the natural-language answer** comes from the LLM.

The **ability to retrieve current weather** comes from the application and its external tools.

These tools can simply be normal functions or programs that perform deterministic tasks.

For example:

```
Weather Tool
Calculator
Database Query
Web Search
Email Service
Payment Service
```

The LLM and these tools together make the application much more capable.

3. Why Does an LLM Need Tools?

An LLM fundamentally generates tokens. It is not a traditional calculator, database, search engine, or operating system.

For a simple calculation, an LLM may still produce the correct result because similar patterns may have appeared extensively during training.

But when reliable computation is required, we can give the model access to an actual calculator or code-execution tool.

This creates an interesting problem.

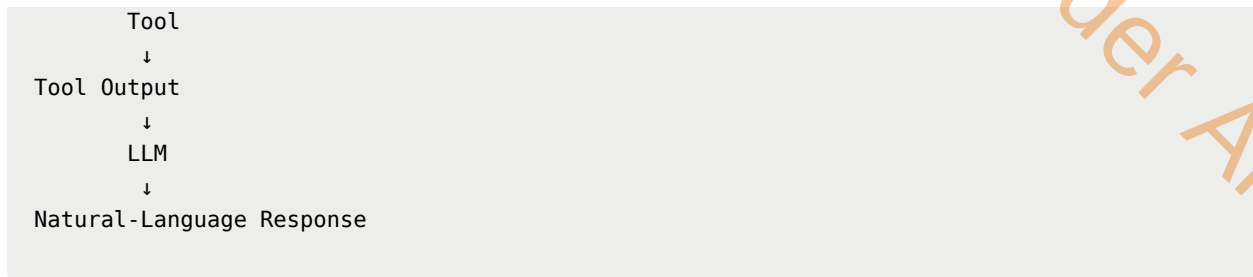
Humans communicate using natural language:

```
Calculate how many * characters are present in this text.
```

But a deterministic program expects structured input.

The LLM can act as the bridge:

```
Natural-Language Input
↓
LLM
↓
Structured Tool Input
↓
```



The LLM is therefore useful not only for generating the final answer but also for understanding what the user wants and preparing the appropriate input for another tool.

4. The Most Important Principle

| The LLM only knows what reaches the LLM.

This simple idea explains a large part of GenAI application engineering.

If the model needs:

Conversation history

→ The relevant conversation must somehow be supplied to it.

Database information

→ The application must retrieve that information and provide it.

Current weather

→ The application needs access to a live weather source.

Reliable calculations

→ The application can provide a calculator or execution tool.

Actions

→ The application must actually execute those actions.

An LLM does not magically have access to every capability of the application surrounding it.

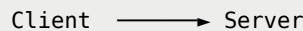
5. Can We Talk Directly to an LLM?

Yes.

We don't need an application such as ChatGPT to communicate with an LLM provider.

Our own application can communicate with the provider through an **API**.

This is similar to normal client-server communication.



```
graph LR; Client --> Server
```

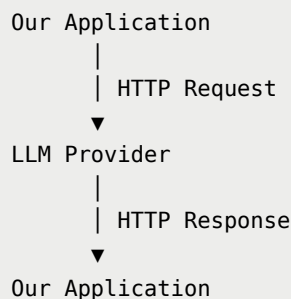
For example:



```
POST /payments
```

with some JSON data.

Communication with an LLM provider follows the same basic idea:



```
graph TD; A[Our Application] -- HTTP Request --> B[LLM Provider]; B -- HTTP Response --> A
```

There is no mysterious special connection required just because AI is involved.

At the infrastructure level, we are primarily making an API request.

6. What Does an LLM API Request Need?

Before making our first request, there are four important things to understand.

6.1 Endpoint

Our application needs to know **where the request should be sent**.

Conceptually:

```
Application
  ↓
API Endpoint
```

The endpoint identifies the service we want to use.

6.2 Authentication

AI models require significant computational resources.

Providers therefore need to identify and authorize applications making requests.

This is usually done using an **API credential**.

Conceptually:

```
Authorization: <credential>
```

The credential identifies and authorizes our application when it communicates with the provider.

| API keys should never be exposed publicly or committed to GitHub.

6.3 Model

A provider may offer multiple models.

Different models can optimize for different requirements such as:

- Cost
- Speed
- Reasoning capability
- Context size
- Multimodality

Therefore, our request usually needs to specify which model we want to use.

Conceptually:

```
{  
  "model": "Model X"  
}
```

6.4 Input

Finally, the model needs something to process.

For example:

```
Summarize this support ticket:  
  
"I was charged twice..."
```

The text eventually gets converted into tokens and processed by the model.

This connects what we learned about LLM internals with real application development:

```
Application  
  ↓  
HTTP/API Request  
  ↓  
Input Text  
  ↓  
Tokens  
  ↓  
Transformer  
  ↓  
Generated Tokens  
  ↓  
API Response  
  ↓  
Application
```

7. Our First FDE Problem

Imagine being deployed as an FDE to a SaaS company's support team.

A customer submits the following ticket:

```
Hi team,  
  
Our production deployment has failed three times since yesterday.  
It looks like the payment-service container keeps restarting.  
Customers are occasionally getting 502 errors.  
  
We already tried restarting the deployment manually but the issue  
returned after approximately twenty minutes.  
  
Can someone investigate urgently?
```

Instead of making someone manually read every large ticket, we want an LLM to summarize it.

Our instruction could be:

```
Summarize this support ticket in 2 lines.
```

A possible response:

```
Production payment-service is repeatedly restarting and causing  
intermittent 502 errors. Manual restarts only temporarily resolve  
the issue.
```

Let's first implement this without Spring or Java.

8. Talking to the Model Using Postman

Step 1 — Create an API Key

Create an API credential from the LLM provider.

The API key will authenticate our application when it communicates with the provider.

Keep the key private.

Step 2 — Create an HTTP Request

In Postman:

New → HTTP Request

Choose:

POST

For the OpenAI Responses API:

<https://api.openai.com/v1/responses>

Conceptually:

```

Postman
  |
  | HTTP POST
  ▼
OpenAI API
  |
  ▼
LLM
  
```

9. Add Authentication

In Postman:

```

Authorization
  ↓
Type: Bearer Token
  
```

Provide the API key.

Conceptually, the HTTP request contains:

```
Authorization: Bearer <API_KEY>
```

The server uses this credential to identify and authorize the request.

10. Specify the Content Type

Since our request body contains JSON:

```
Content-Type: application/json
```

Our request now conceptually looks like:

```
POST /v1/responses  
  
Authorization: Bearer <API_KEY>  
Content-Type: application/json
```

We are authenticated, but we haven't yet told the model what it should process.

11. Create the Request Body

Select:

```
Body → raw → JSON
```

A simple request can look like:

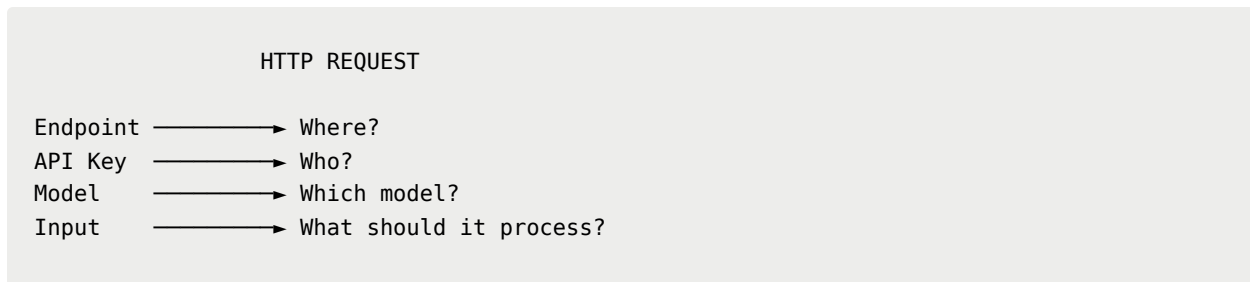
```
{  
  "model": "gpt-5.6-luna",
```

```
"input": "Explain Docker in two simple sentences."
}
```

From first principles, every important part answers a simple question:

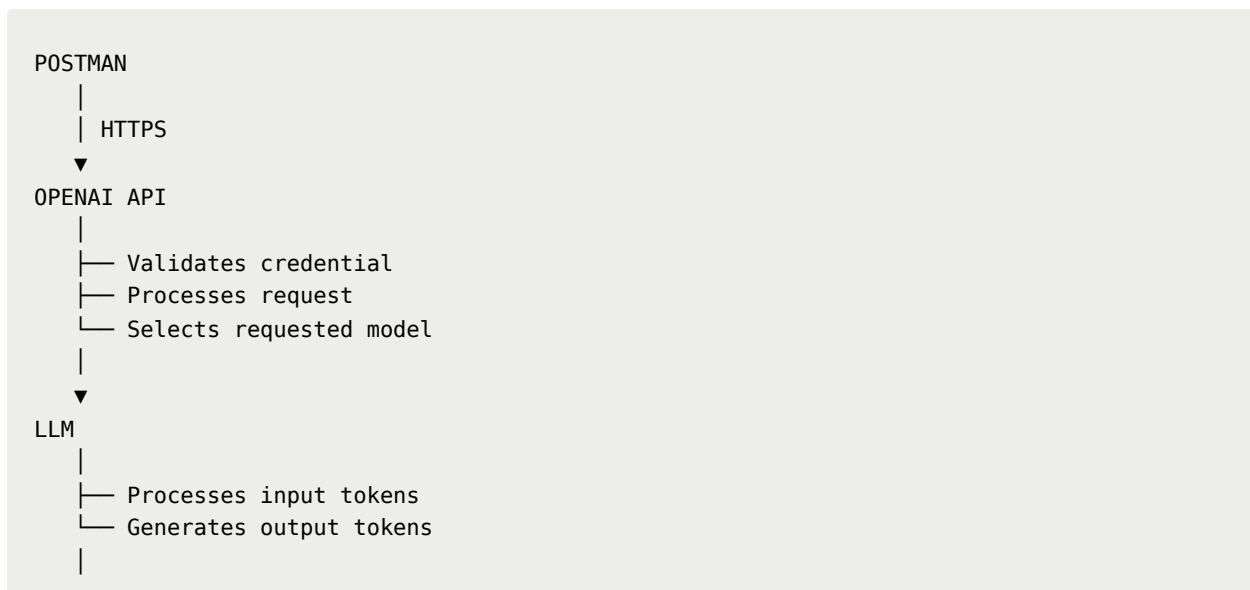
Component	Question
Endpoint	Where should the request go?
API Key	Who is making the request?
Model	Which model should process it?
Input	What should the model process?

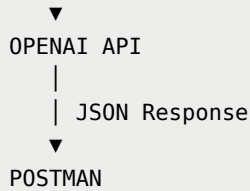
Therefore:



12. What Happens After We Click Send?

The complete flow looks roughly like this:





We have now communicated with an LLM **without using ChatGPT**.

13. Understanding the API Response

An API response contains more than just the generated text.

A simplified conceptual response might look like:

```
{
  "id": "resp_...",
  "object": "response",
  "status": "completed",
  "model": "gpt-5.6-luna",

  "output": [
    {
      "type": "message",
      "role": "assistant",
      "content": [
        {
          "type": "output_text",
          "text": "Docker packages an application..."
        }
      ]
    }
  ],

  "usage": {
    "input_tokens": 15,
    "output_tokens": 30,
  }
}
```

```
"total_tokens": 45
}
}
```

The response can broadly be divided into two categories:

```
RESPONSE
├── Generated Content
│   └── Actual model answer
└── Metadata
    ├── Response ID
    ├── Model
    ├── Status
    ├── Token Usage
    └── Other Information
```

For most applications, the generated content is what eventually reaches the user, while the metadata helps us understand and manage the model interaction.

14. Token Usage Becomes an Engineering Concern

Suppose an API response reports:

```
Input Tokens: 150
Output Tokens: 40
Total Tokens: 190
```

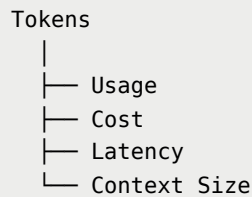
Why does this information matter?

Because tokens represent model usage and computation.

The theoretical concept of:

Tokens

now becomes connected to practical engineering concerns:



This is why understanding tokens is important even when we are simply integrating an existing model into an application.

15. Input Tokens vs Output Tokens

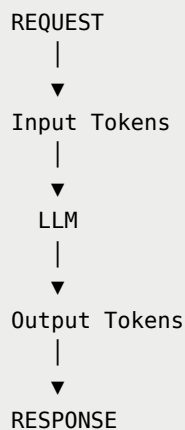
Suppose we send a 10-page customer incident report and ask:

Summarize this incident report.

The model must first process the supplied text.

Those are **input tokens**.

If the model generates a 10-line summary, those newly generated tokens are **output tokens**.



In simple terms:

Input tokens are what the model reads. Output tokens are what the model generates.

16. Moving from Postman to Our Application

Postman proved that communicating with an LLM is fundamentally an API interaction.

But our final goal is not to manually send requests from Postman.

We want our application to do it.

For this series, we will use **Spring AI**.

```
OpenAI / Anthropic / Gemini
    ↓
Actual AI Models

    Spring AI
    ↓
Java abstractions for interacting
with AI models
```

Spring AI does not provide the underlying intelligence.

The model provider provides the model.

Spring AI provides abstractions that make those models easier to integrate into Java applications.

17. Create the Spring Boot Project

For our first example, we only need normal web support and OpenAI model integration.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```


`<dependency>
 <groupId>org.springframework.ai</groupId>
 <artifactId>spring-ai-starter-model-openai</artifactId>
</dependency>`

The OpenAI model starter gives Spring AI the required integration with OpenAI models.

18. Configure the API Key

Remember what we manually provided in Postman:

```
Authorization  
  ↓  
Bearer Token  
  ↓  
API Key
```

Our Spring application needs the same credential.

Instead of manually creating the HTTP authorization header, configure:

```
spring:  
ai:  
openai:  
api-key: ${OPENAI_API_KEY}
```

Then provide the environment variable on the machine:

```
export OPENAI_API_KEY="your-api-key"
```

Avoid doing this:

```
api-key: sk-xxxxxxx
```

Hard-coding credentials is dangerous, especially when the project is pushed to GitHub.

19. Configure the Model

The model can also be configured in `application.yml`.

For example:

```
spring:
  ai:
    openai:
      api-key: ${OPENAI_API_KEY}
    chat:
      model: gpt-5-mini
```

Notice the connection with our Postman request.

Postman

```
{
  "model": "gpt-5-mini"
}
```

Spring AI

```
chat:
  model: gpt-5-mini
```

The conceptual requirement is the same.

Only the place where we configure it has changed.

20. What Have We Replaced So Far?

Our Postman request needed:

API Key
Model
Input

Our Spring configuration now contains:

application.yml

API Key ✓
Model ✓
Input ?

The remaining question is:

| Where will the actual user input come from?

From our Java application.

This is where `ChatClient` enters the picture.

21. Creating a ChatClient

Create a service:

```
@Service
public class SummarizerService {

    private final ChatClient chatClient;

    public SummarizerService(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }
}
```

At this stage, the important idea is simply:

Spring AI gives our application a configured client capable of communicating with the LLM provider.

There is no need to think about the internal Spring mechanics yet.

22. Sending Our First Prompt

Suppose our Postman request contained:

```
{
  "model": "gpt-5-mini",
  "input": "Summarize this support ticket: Payment was deducted but my order is still processing."
}
```

Using Spring AI:

```
String response = chatClient.prompt()
    .user("""
        Summarize this support ticket:

        Payment was deducted but my order is still processing.
        """)
    .call()
    .content();
```

Instead of memorizing this as Spring syntax, understand what each operation represents.

```
prompt()
  ↓
Prepare a model request

user(...)
  ↓
Provide the text for the model
```

call()
↓
Call the configured model

content()
↓
Return the generated text

The syntax is simply an abstraction over the same model interaction we performed manually using Postman.

23. Building the Summarizer Service

Now make the support ticket dynamic.

```
@Service
public class SummarizerService {

    private final ChatClient chatClient;

    public SummarizerService(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }

    public String summarize(String ticket) {

        return chatClient.prompt()
            .user(
                "Summarize this support ticket in two sentences:\n\n"
                + ticket
            )
            .call()
            .content();
    }
}
```

```
}  
}
```

The method now accepts any support ticket and sends it to the configured model.

24. Expose the Summarizer as an API

Create a REST controller:

```
@RestController  
@RequestMapping("/api")  
public class SummarizerController {  
  
    private final SummarizerService summarizerService;  
  
    public SummarizerController(  
        SummarizerService summarizerService) {  
  
        this.summarizerService = summarizerService;  
    }  
  
    @PostMapping("/summarize")  
    public String summarize(  
        @RequestBody SummarizeRequest request) {  
  
        return summarizerService.summarize(request.text());  
    }  
}
```

Now our application exposes:

```
POST http://localhost:8080/api/summarize
```

Example request:

```
{
  "text": "Payment was deducted three days ago but the order
still shows Payment Processing. I contacted support twice and
need the product tomorrow for my daughter's birthday."
}
```

25. Complete Request Flow

The controller receives:

```
request.text()
```

which gives us the customer's text.

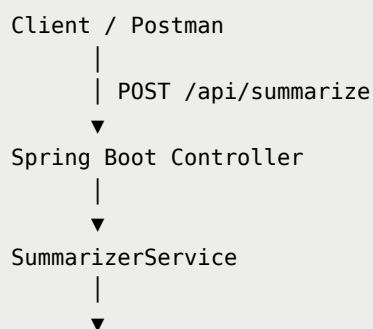
It is passed to:

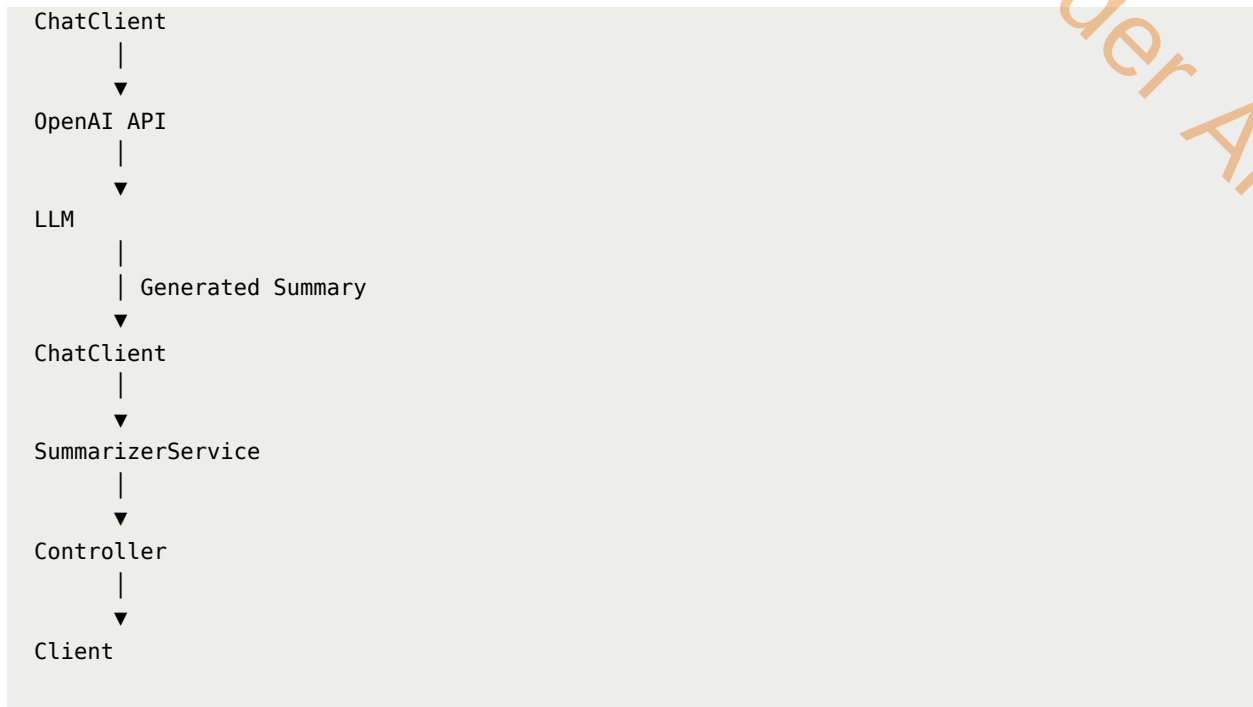
```
summarizerService.summarize(request.text())
```

The service then executes:

```
chatClient.prompt()
    .user(...)
    .call()
    .content();
```

The complete architecture becomes:





We now have our first real GenAI-powered API.

26. Why Use Spring AI Instead of Calling the HTTP API Directly?

Technically, we don't need Spring AI.

We could use `RestClient` and manually create the same HTTP request we used in Postman.

For example, we could manually send:

```

{
  "model": "...",
  "input": "..."
}
  
```

That would work.

The value of an abstraction such as Spring AI becomes clearer as the application grows.

Later we may need:

- Multiple messages
- Conversation memory
- Streaming
- Structured outputs
- Tool calling
- RAG
- Multiple model providers
- Observability

If we manually implement provider-specific HTTP requests and response parsing for everything, our application can quickly become tightly coupled to one provider's API format.

Spring AI gives us a higher-level abstraction for these interactions.

For a tiny application, this may save only a little code.

For a larger GenAI application, the abstraction becomes much more valuable.

27. Getting More Than Just the Generated Text

So far we used:

```
.content()
```

because we only wanted the generated text.

Sometimes we need access to richer information about the model response.

Spring AI allows us to obtain the complete response:

```
ChatResponse response = chatClient.prompt()
    .user(prompt)
```

```
.call()  
.chatResponse();
```

This becomes useful when we want more than the final text and need information associated with the model interaction.

28. The Next Problem: Instructions vs User Data

Our current summarizer contains:

```
.user(  
    "Summarize this support ticket in two sentences:\n\n"  
    + ticket  
)
```

Notice what we have done.

Two different kinds of information are mixed together:

```
Application Instruction  
    +  
Customer's Actual Data  
    ↓  
Single User Message
```

For example:

```
Summarize this support ticket in two sentences:  
  
Payment was deducted three days ago...
```

But these two pieces of information have different purposes.

```
"Summarize this support ticket in two sentences"  
    ↓  
Application Instruction
```

"Payment was deducted..."

↓

User Data

Should instructions coming from our application and data coming from the end user really be treated as exactly the same thing?

This brings us to an important concept in LLM applications: